

CSC 3570 – IT Security

Lab Portfolio

Team 1 — Project: Pompeii Strategies

Anthony Lamantia · Adam Campbell · Thomas Herbert

Tennessee Tech University · CEROC CyberRange

Project Overview

For this semester-long project, our three-person team took on the role of an incoming IT department for “Pompeii Strategies,” a fictional company whose network had been left undocumented by a previous administrator. Working inside Tennessee Tech's CEROC CyberRange, we inherited four virtual machines: a Windows 10 workstation, a Windows Server 2019 domain controller, an Ubuntu web server, and an OpenWrt router/firewall, and progressively brought the environment up to a modern security baseline across eight lab modules.

Each lab built on the last: we started by simply getting the machines talking to each other, then layered on malware protection, centralized identity and policy, firewall segmentation, configuration automation, hardened remote access, and finally automated intrusion prevention. Every change was tracked in a running changelog and validated with screenshots, consistent with the change-management practices expected of a real production environment.

Environment at a Glance

Domain	pompeii01.net (Active Directory)
Domain Controller	Windows Server 2019 — 192.168.1.121
Workstation	Windows 10 — 192.168.1.122
Web Server	Ubuntu (Apache2, PHP) — 192.168.1.130
Firewall / Router	OpenWrt — WAN / LAN / DMZ zones, 192.168.1.1
Platform	Tennessee Tech CEROC CyberRange (LXD-based)

Labs Covered in This Write-Up

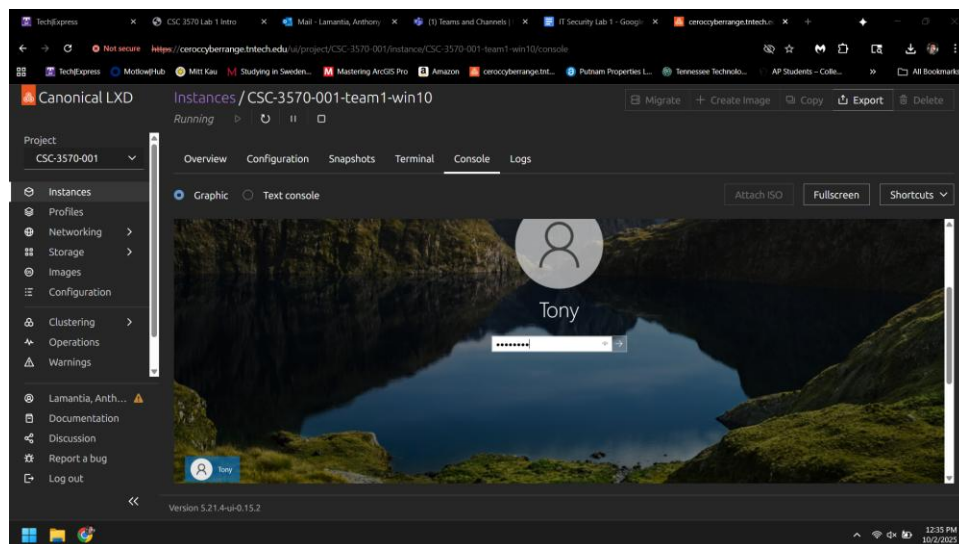
- Lab 1 — Introduction & Infrastructure Setup
- Lab 2 — Baseline Security (AV/malware scanning, Lynis, Flan Scan)
- Lab 3 — User Authentication & Group Policy
- Lab 4 — Configuration Management (Docker & SaltStack)
- Lab 5 — Firewalls (OpenWrt & UFW)
- Lab 7 — SSH & SSH Keys
- Lab 8 — Intrusion Detection/Prevention (Fail2ban)

Lab 1 — Introduction & Infrastructure Setup

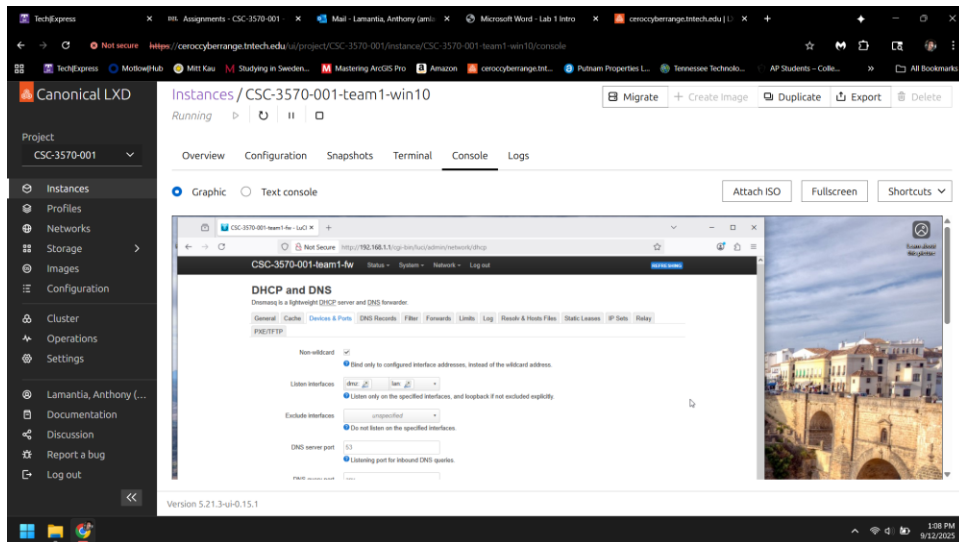
Objective: Stand up the inherited environment, establish baseline access to every machine, and get the network addressing and domain groundwork in place before any hardening work could begin.

What We Did

- Logged into each inherited VM (Windows 10 workstation, Windows Server 2019, Ubuntu web server, OpenWrt router) for the first time and confirmed credentials.
- Updated the Windows 10 workstation and created a new local admin account (“Tony”) separate from the built-in default, per policy.
- Assigned static IPs across the network: workstation at 192.168.1.122, domain controller at 192.168.1.121, with the OpenWrt router as gateway/DNS at 192.168.1.1.
- Set the OpenWrt router's initial admin password and enabled the DMZ interface under DHCP/DNS → Listen Interfaces so the web server's zone could resolve DNS.
- Installed Active Directory Domain Services and DNS on the Windows Server 2019 box, then promoted it to a domain controller for a new forest, pompeii01.net.
- Joined the Windows 10 workstation to the new domain and confirmed logon with domain credentials.
- On the Ubuntu server, created sudo accounts for each team member and ran system updates.



Logging into the Windows 10 workstation with the new local admin account (“Tony”) via the CyberRange console.



Enabling the DMZ interface under DHCP and DNS → Listen Interfaces on the OpenWrt router.

Changelog Excerpt

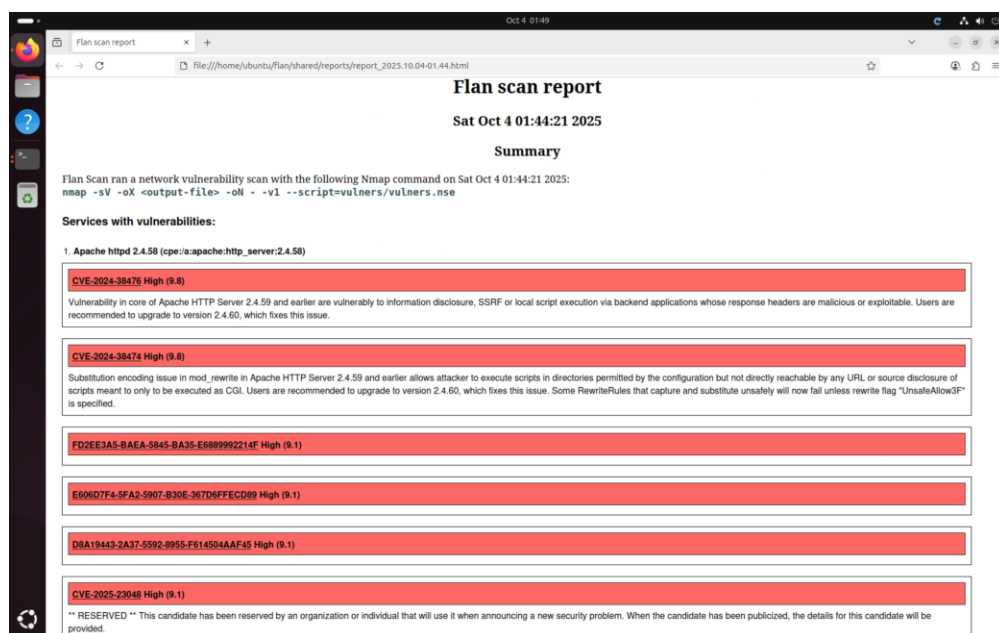
9/22	Forced password requirement on the Administrator account via “net user Administrator /passwordreq:yes”
9/22	Set static IP 192.168.1.121/25 on the domain controller; gateway/DNS 192.168.1.1
9/22	Installed AD DS and DNS roles; promoted the server to a domain controller (pompeii01.net)
9/23	Confirmed 192.168.1.121 as the workstation's DNS server for domain lookups
9/23	Joined the workstation to the domain and verified domain-credential logon

Lab 2 — Baseline Security

Objective: Establish an anti-malware and vulnerability-scanning baseline across every OS in the environment, and start using automated tools (Lynis, Flan Scan) to surface issues before an attacker does.

What We Did

- Added a shared “devops” account to both the domain (with Domain Admin membership) and the Ubuntu web server, then ran Malwarebytes quick scans on the Windows Server and Windows 10 machines.
- Installed ClamAV on the Ubuntu web server and ran a recursive scan against /usr/local/bin/.
- Installed and ran Lynis on the web server, then filtered /var/log/lynis.log for hardening suggestions.
- Installed Flan Scan (Cloudflare's Nmap-based vulnerability scanner) on the jumpbox and scanned two in-scope hosts: the Windows Server 2016 machine (192.168.1.121) and a branch-office Linux server (10.0.14.4).



Flan Scan HTML report identifying an outdated Apache httpd 2.4.58 install and associated CVEs.

Vulnerabilities Identified (Flan Scan)

- CVE-2024-38476 — Apache HTTP Server 2.4.58 (High, 9.8): core response-header handling could allow information disclosure, SSRF, or local script execution. Remediation: upgrade to Apache 2.4.60+.
- CVE-2024-38474 — Apache HTTP Server 2.4.59 (High, 9.8): mod_rewrite substitution issue could let an attacker execute scripts outside intended directories. Remediation: upgrade to 2.4.60 and avoid unsafe RewriteRule flags.
- CVE-2025-23048 (High, 9.1): reserved CVE for a pending Apache disclosure. Remediation: monitor Apache security bulletins and patch promptly once details are published.

Top Lynis Hardening Suggestions

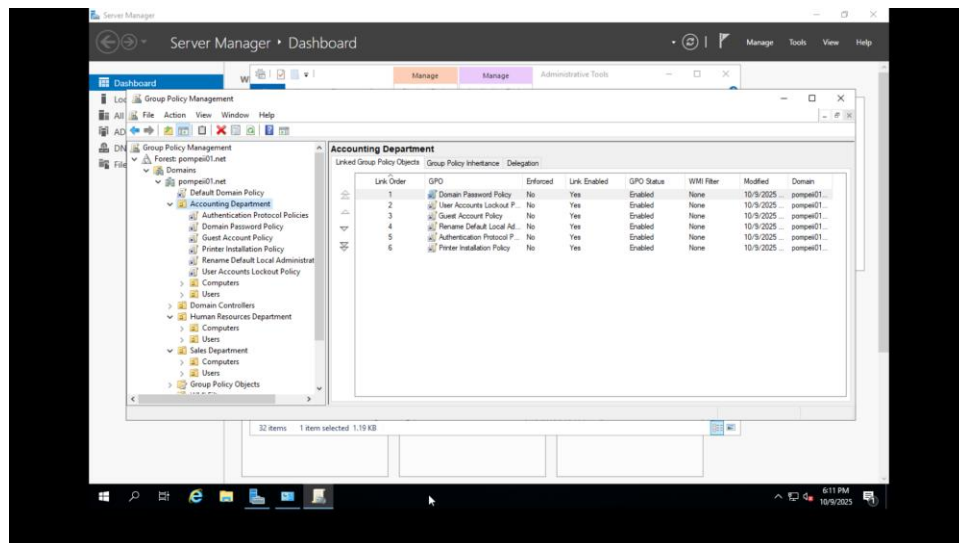
- Install fail2ban to block IPs after repeated failed logins and mitigate brute-force attacks against SSH/web services (this became the focus of Lab 8).
- Set a GRUB bootloader password to stop anyone with physical/console access from booting into single-user mode.
- Disable unused or unnecessary services to shrink the overall attack surface.

Lab 3 — User Authentication & Group Policy

Objective: Build out the Active Directory organizational structure, add real user accounts and security groups, and enforce security settings domain-wide with Group Policy.

What We Did

- Created three top-level Organizational Units: Accounting, Sales, and Human Resources (each with Users and Computers sub-OUs, inside the pompeii01.net domain).
- Created department security groups (Finance under Accounting, Records under HR) and populated all six specified employee accounts in their correct OUs and groups.
- Shared a “Budget” folder on the workstation and restricted access to only the Finance security group, then validated the permission boundary.
- Built six Group Policy Objects in the Accounting Department OU: a Domain Password Policy (5-password history, 90/30-day max/min age, 9-character minimum, complexity enabled, reversible encryption disabled), an Account Lockout Policy, a Guest Account Policy (disabled), a policy renaming the default local Administrator account, an Authentication Protocol Policy (NTLMv2-only, LM hash storage disabled), and a Printer Installation Policy blocking driver installs by standard users.
- Linked all six GPOs to the Sales and Human Resources OUs so the same baseline applied company-wide, not just to Accounting.
- On the Ubuntu web server, created a “dev” group, added the devops user to it, and applied ownership/permission (chown/chmod) restrictions to a new /devops directory so only that group could read/write/execute it.



Group Policy Management console showing all six GPOs linked and enabled under the Accounting Department OU in pompeii01.net.

Deliverable Highlights

- Group Policy exists to centrally enforce and audit consistent security and configuration settings across every machine in a domain, rather than configuring each computer by hand.
- An Account Lockout Policy on a machine with RDP enabled opens the door to a denial-of-service style attack: an attacker can deliberately fail logins to lock out legitimate user accounts.

- PowerShell is preferred for Windows Server administration because it is scriptable and repeatable — tasks can be automated, version-controlled, and run identically across many servers instead of relying on manual GUI clicks.

Lab 4 — Configuration Management (Docker & SaltStack)

Objective: Learn to manage system configuration at scale with SaltStack — pushing packages, files, and user accounts to a minion from a central Salt Master — running inside a Dockerized lab environment.

What We Did

- Cloned the course's Docker-based Salt lab and used Task (a Makefile-style runner) to start the environment and drop into a root shell on the salt-master container.
- Verified master-to-minion connectivity with `salt '*' test.ping`, then built a top.sls/pillar structure to push a package (vim) to all minions using Jinja templating pulled from pillar data.
- Extended the state tree to install Apache2 and create two new Linux users (fred and devops) via Salt states, including generating and passing a proper password hash for devops after an initial typo required troubleshooting.
- Used Salt's file.managed state to push a test file from the master to `/home/devops/` on the minion, then verified it landed correctly and that both new user accounts could log in with the expected passwords.

```
Comment: User fred is present and up to date
Started: 03:04:53.666804
Duration: 2.805 ms
Changes:
-----
ID: devops
Function: user.present
Result: True
Comment: User devops is present and up to date
Started: 03:04:53.667969
Duration: 2.303 ms
Changes:
-----
ID: /home/devops/test
Function: file.managed
Result: True
Comment: File /home/devops/test updated
Started: 03:04:53.679596
Duration: 95.754 ms
Changes:
-----
diff:
  new file
group:
  devops
mode:
  0644
user:
  devops

Summary for 1904c419e46d
-----
Succeeded: 5 (changed=1)
Failed: 0
-----
Total states run: 5
Total run time: 162.436 ms
root@salt-master:/srv/salt# cd ..
root@salt-master:/srv# cd ..
root@salt-master:/# exit
exit
user@c3c-3570-001-teaml-dmz-ubuntu:/saltlab/csc3570-salt-lab$ sudo task connect-minion
[sudo] password for user:
task: connect-minion docker-compose exec salt-minion bash
root@1904c419e46d:/# ls -l /home/devops/
total 4
-rw-r--r-- 1 devops devops 13 Oct 27 03:04 test
root@1904c419e46d:/# cat /home/devops/test
hello world!
root@1904c419e46d:/#
```

Salt state.apply summary on the minion showing all five states succeeded — user creation, package install, and file delivery — verified via docker-compose exec.

Deliverable Highlights

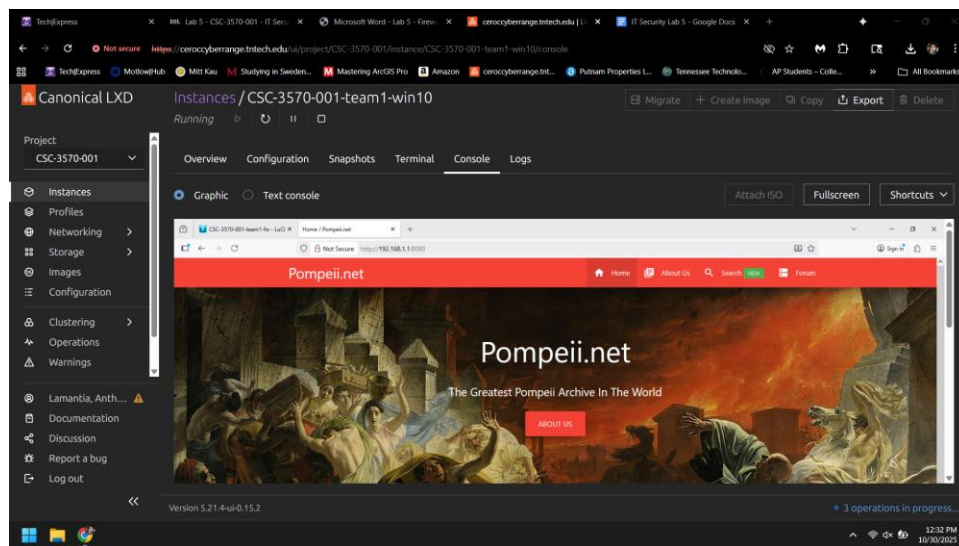
- A configuration management tool like SaltStack pays off through automation, consistency, and the ability to manage large fleets of systems efficiently instead of configuring each host by hand.
- A compromised Salt Master is a high-value target: an attacker who controls it can push malicious commands or configurations to every connected minion at once.
- A state describes what a system should look like or what actions to take; a pillar is secure, minion-specific data (like credentials) stored on the master and delivered only to the systems that need it.
- Ansible is a comparable tool, but differs in being agentless and push-based from a control node, versus SaltStack's agent (minion) based model.

Lab 5 — Firewalls

Objective: Segment the network into WAN, LAN, and DMZ zones on the OpenWrt firewall, expose only the web server to the internet, and add a second layer of host-based filtering with UFW on the Ubuntu server.

What We Did

- Configured port forwarding so the WAN zone forwards port 80 to the web server (192.168.1.130) and the LAN zone can reach it internally on port 8080, then confirmed the Pompeii.net site loaded from inside the network.
- Built a full traffic-rule set across all three zones: WAN allows inbound HTTP/FTP to the web server only; LAN allows SSH/FTP/HTTP/HTTPS/DNS outbound; DMZ is allowed to reach update repositories, send RDP to the workstation, and resolve DNS.
- Added explicit default-deny rules from LAN and DMZ to every other zone, so nothing is implicitly trusted beyond what was explicitly allowed.
- Installed and enabled UFW on the Ubuntu web server as host-based defense-in-depth: denied HTTP/SSH from the Domain Controller's IP specifically, allowed HTTP from anywhere, and restricted SSH to only the workstation's IP.
- Verified the layered configuration by browsing to the web server from both the DC and the workstation and confirming ufw status matched the intended rule set.



Confirming the port-forwarding rule works: the Pompeii.net homepage loads through the OpenWrt LAN-zone forward on port 8080.

Deliverable Highlights

- A DMZ isolates internet-facing services from the internal network, so a compromise of a public-facing host doesn't automatically expose private company resources.
- Servers that must be reachable from the public internet: web, mail, FTP, DNS, each belonging in the DMZ so a breach there is contained away from internal data.
- Key ports: SSH 22/TCP, RDP 3389/TCP, HTTP 80/TCP, HTTPS 443/TCP, DNS 53/TCP+UDP.
- Egress filtering matters as much as ingress: it stops malware from phoning home, blocks data exfiltration, and prevents compromised hosts from reaching command-and-control infrastructure.
- Our firewall is zone-based (WAN/LAN/DMZ) rather than address-by-address, and NAT lets every internal host share the router's single public IP; conserving IPv4 space and hiding internal topology from the outside world.

Lab 7 — SSH & SSH Keys

Objective: Move remote administration of the web server from password authentication to public-key authentication, and understand SSH tunneling along the way.

What We Did

- Installed and confirmed SSH on the Ubuntu web server, then temporarily re-enabled PasswordAuthentication in sshd_config to bootstrap the first connection from the Windows 10 workstation.
- Generated an RSA key pair on the workstation and copied the public key to the web server's ~/.ssh/authorized_keys, then confirmed key-based login worked with ssh -i.
- Built an SSH client config (~/.ssh/config) with a friendly host alias pointing at the web server's IP, key file, and username, so future connections were a single short command.
- Edited sshd_config a second time to disable password authentication entirely, restarted sshd, and re-tested to confirm only key-based logins were accepted going forward.
- Demonstrated SSH tunneling by forwarding local port 80 to the web server's port 80 (ssh <alias> -L 80:192.168.1.130:80 -N) and loading the Pompeii site through localhost, and used the w command to confirm active sessions on the remote host.

```
sudo] password for user:
sudo: cd: command not found
sudo: "cd" is a shell built-in command, it cannot be run directly.
sudo: the -s option may be used to run a privileged shell.
sudo: the -D option may be used to run a command in a specific directory.
user@CSC-3570-001-team1-dmz-ubuntu:~$ cd
user@CSC-3570-001-team1-dmz-ubuntu:~$ cd .ssh/
user@CSC-3570-001-team1-dmz-ubuntu:~/.ssh$ nano config
user@CSC-3570-001-team1-dmz-ubuntu:~/.ssh$ user@CSC-3570-001-team1-dmz-ubuntu:~/.ssh$ nano
user@CSC-3570-001-team1-dmz-ubuntu:~/.ssh$ ssh user
/home/user/.ssh/config: line 4: Bad configuration option: identityfile
/home/user/.ssh/config: terminating, 1 bad configuration options
user@CSC-3570-001-team1-dmz-ubuntu:~/.ssh$ nano config
user@CSC-3570-001-team1-dmz-ubuntu:~/.ssh$ ssh user
The authenticity of host '192.168.1.130 (192.168.1.130)' can't be established.
ED25519 key fingerprint is SHA256:yKH152BPE12gHlydu0EskSI9H1ONux9UT6QhPYDm4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.130' (ED25519) to the list of known hosts.
No such identity: /home/user/.ssh/id_ed25519: No such file or directory
user@192.168.1.130's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-87-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Nov 20 18:58:52 UTC 2025

System load:          0.0
Usage of /:            11.7% of 37.70GB
Memory usage:         14%
Swap usage:           0%
Processes:            155
Users logged in:      1
IPv4 address for enp5s0: 192.168.1.130
IPv6 address for enp5s0: fd42:54ad:a130:fe4c:216:3eff:fe80:2e72

Expanded Security Maintenance for Applications is not enabled.

19 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Thu Nov 20 18:48:52 2025 from 192.168.1.122
user@CSC-3570-001-team1-dmz-ubuntu:~$
```

First SSH connection from the workstation to the web server, accepting the host key and authenticating before the switch to key-only auth.

Deliverable Highlights

- A VPN (IPsec/WireGuard/OpenVPN) secures and routes all of a device's traffic as if it were on the remote network, whereas an SSH tunnel secures and forwards just one specific application's connection.

- A common SSH tunneling use case is reaching an internal service (a database or internal web app) that isn't otherwise exposed, with all traffic riding inside the encrypted SSH session.
- SSH keys authenticate via a public/private key pair that's cryptographically infeasible to brute-force, and the private key itself is never transmitted over the network.

Lab 8 — Intrusion Detection/Prevention (Fail2ban)

Objective: Deploy Fail2ban on the web server to automatically detect and block brute-force SSH attempts, closing the loop on the recommendation Lynis surfaced back in Lab 2.

What We Did

- Installed Fail2ban on the Ubuntu web server and copied jail.conf to jail.local (the standard practice so local overrides survive package updates), setting ignoreip to the loopback address.
- Created a dedicated sshd.local jail under /etc/fail2ban/jail.d/ with mode=aggressive, port=ssh, the iptables-multiport action, logpath pointed at /var/log/auth.log, maxretry=3, and a 600-second bantime.
- Enabled and started the fail2ban service, then triggered a real test from another machine without SSH keys configured in which the connection error switched from “denied (publickey)” to “connection refused” on the fourth attempt, confirming the ban engaged.
- Confirmed the ban in /var/log/fail2ban and practiced manually lifting it early with fail2ban-client set sshd unbanip.
- Drafted a second jail intended to protect an Apache login form the same way (noting the site has no login page to actually exercise against), and researched Snort as a comparison point to fail2ban.

[illegible]

Installing Fail2ban on the Ubuntu web server.

```

Dec 4 18:34

ubuntu@CSC-3570-001-team1-JumpBox:~$ ssh user@192.168.1.130
ssh: connect to host 192.168.1.130 port 22: Connection timed out
ubuntu@CSC-3570-001-team1-JumpBox:~$ ssh user@192.168.1.130
The authenticity of host '192.168.1.130 (192.168.1.130)' can't be established.
ED25519 key fingerprint is SHA256:YKH1520P1ZgHLYduEskK91HmUNuX9U7tGqHPwDn4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.130' (ED25519) to the list of known hosts.
user@192.168.1.130: Permission denied (publickey).
ubuntu@CSC-3570-001-team1-JumpBox:~$ ssh user@192.168.1.130
user@192.168.1.130: Permission denied (publickey).
ubuntu@CSC-3570-001-team1-JumpBox:~$ ssh user@192.168.1.130
user@192.168.1.130: Permission denied (publickey).
ubuntu@CSC-3570-001-team1-JumpBox:~$ ssh user@192.168.1.130
ssh: connect to host 192.168.1.130 port 22: Connection refused
ubuntu@CSC-3570-001-team1-JumpBox:~$

```

Fail2ban in action from a second machine: repeated failed public-key attempts flip to “Connection refused” once the ban threshold is hit.

Fail2ban vs. Snort (Research Question)

Fail2ban is a reactive, log-based host protection tool: it watches log files (like `/var/log/auth.log`) for patterns of failed authentication and temporarily bans the offending IP at the firewall level, making it well suited to stopping brute-force login attempts against services like SSH or web-based logins. Snort, by contrast, is a full network intrusion detection/prevention system that inspects live network traffic against a much broader rule set; covering exploit signatures, protocol anomalies, port scans, and more across the whole network rather than one host's logs. In practice the two are complementary: Fail2ban is lightweight, host-focused brute-force protection, while Snort provides deeper, network-wide visibility into a much larger range of attack types.